

Programação para Dispositivos Móveis

Promises, Fetch, async/await

Universidade Federal do Rio Grande do Norte

Introdução

- Comunicação assíncrona.
 - Processamento assíncrono permite que o código continue sendo executado enquanto espera por uma tarefa longa, como o retorno de uma requisição a um servidor.
 - Exemplo de tarefas comuns: requisições HTTP, leitura/escrita de arquivos, ou temporizadores.
- Por que usar?
 - Melhora a performance e a experiência do usuário.
 - Evita o blocking da thread principal.

Promises

- Uma Promise é um objeto que representa uma operação assíncrona e seu eventual resultado (sucesso ou falha).
- Estados de uma Promise:
 - Pending: Operação ainda em execução.
 - Fulfilled: Operação concluída com sucesso.
 - Rejected: Operação falhou.
- Vantagens:
 - Facilita o encadeamento de operações assíncronas.
 - Melhora a legibilidade em comparação ao uso de callbacks.

Promisses

```
const minhaPromise = new Promise((resolve, reject) => {  
  const sucesso = true; // Simulação  
  if (sucesso) {  
    resolve("Operação concluída com sucesso!");  
  } else {  
    reject("Houve um erro.");  
  }  
});
```

```
minhaPromise  
  .then((resultado) => console.log(resultado))  
  .catch((erro) => console.error(erro));
```

Encadeamento de Promises

- Útil para executar operações assíncronas sequenciais.
- Usa o método `.then()` em sequência.

```
fetch("https://api.exemplo.com/dados")
  .then((response) => response.json())
  .then((data) => {
    console.log("Dados recebidos:", data);
    return fetch("https://api.exemplo.com/mais-dados");
  })
  .then((response) => response.json())
  .then((moreData) => console.log("Mais dados:", moreData))
  .catch((error) => console.error("Erro:", error));
```

Async/Await

- Introduzido no ES2017 (ES8).
- É sintaxe simplificada para trabalhar com Promises.
- Usa as palavras-chave `async` e `await` para melhorar a legibilidade.
- Vantagens:
 - Código mais limpo e estruturado.
 - Facilita o tratamento de erros com `try...catch`.

Async/Await

```
async function buscarDados() {  
  try {  
    const response = await fetch("https://api.exemplo.com/dados");  
    const data = await response.json();  
    console.log("Dados recebidos:", data);  
  } catch (error) {  
    console.error("Erro ao buscar dados:", error);  
  }  
}  
  
buscarDados();
```

Comparação entre Promises e Async/Await

- Promises:

- Vantagem: Encadeamento de múltiplas operações.
- Desvantagem: Pode ficar difícil de ler e manter quando muito aninhado.

- Async/Await:

- Vantagem: Código mais próximo de uma execução síncrona.
- Desvantagem: Necessita que toda a função seja marcada como `async`.

Comparação entre Promises e Async/Await

```
// Usando Promises
fetch('https://api.exemplo.com')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

// Usando Async/Await
async function fetchData() {
  try {
    const response = await fetch('https://api.exemplo.com');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
fetchData();
```

Fetch

- API nativa do JavaScript para fazer requisições HTTP.
- Retorna uma Promise.

```
fetch("https://api.exemplo.com")  
  .then((response) => response.json())  
  .then((data) => console.log(data))  
  .catch((error) => console.error("Erro:", error));
```

Fetch

- Métodos HTTP (GET, POST, PUT, DELETE).
- Cabeçalhos personalizados.
- Envio de corpo (body).

```
const dados = { nome: "João", idade: 30 };

fetch("https://api.exemplo.com/cadastrar", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dados),
})
  .then((response) => response.json())
  .then((data) => console.log("Resposta:", data))
  .catch((error) => console.error("Erro:", error));
```

Tratamento de Erros

- Garantir que erros sejam tratados para evitar falhas não controladas.

```
fetch("https://api.exemplo.com/dados")
  .then((response) => {
    if (!response.ok) throw new Error("Erro na requisição");
    return response.json();
  })
  .then((data) => console.log(data))
  .catch((error) => console.error("Erro capturado:", error));
```

Tratamento de Erros

- Garantir que erros sejam tratados para evitar falhas não controladas.

```
async function buscarDados() {  
  try {  
    const response = await fetch("https://api.exemplo.com/dados");  
    if (!response.ok) throw new Error("Erro na requisição");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error("Erro capturado:", error);  
  }  
}
```

Referências

- https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function